PROCÉDURE · MON INFRASTRUCTURE HAUTE DISPONIBILITÉ · BASES DE DONNÉES ET HA

MO-2 — Répartiteur de charge HAProxy

Installer le répartiteur HAProxy : vérifications d'état, répartition HTTP (WordPress, GLPI) et accès SQL à la grappe.

> 🗂️ Document de mon dossier E6 (BTS SIO SISR), tel que je l'ai rédigé pendant la mise en place de mon laboratoire. Version du 10/10/2026. Les adresses sont celles de mon laboratoire : adapte-les au tien.

Installation, vérifications d'état, répartition HTTP (WordPress, GLPI) et SQL.

> 🔐 Aucun mot de passe ni clé dans ce document : les secrets sont remplacés par des marque-places.
> ⚠️ **Écarts avec le labo réel** (configuration relevée le 01/10/2026, voir le DAT §7.2) : HAProxy contrôle ses serveurs **toutes les 2 s** (`inter 2s fall 3 rise 2`), le contrôle SQL est un simple test de port (`tcp-check` , pas `mysql-check`), et il n'y a **ni page de statistiques (8404) ni en-tête `X-Served-By`** : les passages qui les citent décrivent une cible, pas l'existant. adm-01 est le contrôleur de domaine Windows : les commandes notées « sur adm-01 » se jouent depuis une session du bastion Guacamole.

HAProxy — Installation

Fiche

Élément	Valeur
VM	haproxy
IP	10.0.10.2/24, passerelle 10.0.10.254
VLAN	301 — vWEB
OS	Debian 12
Logiciel	HAProxy 2.6 (paquet Debian)
Compte d'administration	`haproxyadmin` (**pas** `haproxy` — voir ci-dessous)
Configuration	`configs/haproxy/haproxy.cfg.example` (dépôt GitHub)

Compte d'administration : ne pas l'appeler « haproxy »

Lors de l'installation de Debian, l'installateur demande de créer un utilisateur. Le nommer `haproxy` crée un compte **humain** (uid ≥ 1000, shell, home) qui **entre en conflit** avec le compte **système** que le paquet `haproxy` veut créer pour faire tourner le service. Résultat : erreurs à l'installation du paquet ou service lancé avec un compte doté d'un shell — à éviter.

Règle : compte humain `haproxyadmin` , compte système `haproxy` laissé au paquet. Détail : incident INC-08.

```
# SUR haproxy — vérification
getent passwd haproxy        # attendu : uid < 1000, shell /usr/sbin/nologin
```

Installation

SUR haproxy

```
sudo ./scripts/install/install-haproxy.sh           # simulation
sudo ./scripts/install/install-haproxy.sh --apply   # exécution
```

Ou manuellement :

```
sudo apt update
sudo apt install -y haproxy mariadb-client socat curl
haproxy -v
```

Compte de health-check SQL

SUR bdd-01 (répliqué sur les 3 nœuds)

```
-- sudo mariadb
CREATE USER 'haproxy_check'@'10.0.10.2';
-- Aucun GRANT : le compte peut seulement ouvrir une session.
```

Configuration

SUR haproxy

```
sudo cp /etc/haproxy/haproxy.cfg /etc/haproxy/haproxy.cfg.orig
sudo cp configs/haproxy/haproxy.cfg.example /etc/haproxy/haproxy.cfg
sudo nano /etc/haproxy/haproxy.cfg                # remplacer <PASSWORD> (page de stats)
sudo haproxy -c -f /etc/haproxy/haproxy.cfg       # "Configuration file is valid"
sudo systemctl enable --now haproxy
sudo systemctl reload haproxy
```

`reload` applique la nouvelle configuration **sans couper** les connexions en cours ; `restart` les coupe. Toujours lancer `haproxy -c` avant.

Vérification

```
# SUR haproxy
./scripts/validation/check-haproxy.sh
sudo ss -ltnp | grep haproxy        # 80, 3306, 8080, 8404 sur 10.0.10.2
```

Page de statistiques depuis adm-01 : `http://10.0.10.2:8404/stats` — capture attendue `evidence/screenshots/05-haproxy-status.png`.

Suite : sql-load-balancing.md, wordpress-load-balancing.md

HAProxy — Health-checks

Un health-check est un test périodique qui décide si un serveur reçoit du trafic. **Sans health-check, il n'y a pas de haute disponibilité** : HAProxy continuerait d'envoyer des requêtes vers un serveur mort.

Paramètres

Paramètre	Valeur	Effet
`inter 3s`	3 secondes	Intervalle entre deux tests
`fall 3`	3 échecs	Serveur déclaré **DOWN** après ~9 s
`rise 2`	2 succès	Serveur déclaré **UP** après ~6 s
`timeout check 3s`	3 secondes	Délai maximum d'une réponse au test

Compromis : des valeurs plus basses détectent plus vite une panne mais risquent des **faux positifs** (un serveur lent retiré à tort) et ajoutent de la charge.

Checks par service

Backend	Type	Ce qui est testé	Ce qui n'est pas testé
`be_wordpress`	HTTP GET `/wp-login.php`, attendu 2xx/3xx	Apache, PHP, accès base	Contenu de la page
`be_glpi`	HTTP GET `/`, attendu 2xx/3xx	Apache, PHP, routage GLPI	—
`galera_cluster`	`mysql-check` (handshake + login)	MariaDB démarré et joignable	**État Galera** (`Synced`, `Primary`)

Limite du check SQL et amélioration

Un nœud peut **accepter une connexion** tout en étant inutilisable :

- `Donor/Desynced` pendant un SST rsync (bloqué en lecture/écriture) ;
- `non-Primary` après une perte de quorum (`WSREP has not yet prepared node for application use`).

`mysql-check` le considère sain. **Amélioration (roadmap)** : check *wsrep-aware* via un petit service HTTP sur chaque nœud (ex. script `clustercheck` exposé sur le port 9200) qui renvoie `200` uniquement si `wsrep_local_state = 4` (Synced) :

```
# Exemple — amélioration future, NON déployée
listen galera_cluster
    bind 10.0.10.2:3306
    mode tcp
    balance roundrobin
    option httpchk
    http-check expect status 200
    default-server port 9200 inter 3s fall 3 rise 2 on-marked-down shutdown-sessions
    server bdd-01 10.0.20.1:3306 check
    server bdd-02 10.0.20.2:3306 check
    server bdd-03 10.0.20.3:3306 check
```

Observer les checks

```
# SUR haproxy — journal en direct
sudo journalctl -u haproxy -f
# Exemple de ligne lors d'une panne :
#   Server be_wordpress/web-01 is DOWN, reason: Layer4 connection problem ...

# État instantané
./scripts/validation/check-haproxy.sh
echo "show servers state" | sudo socat stdio /run/haproxy/admin.sock
```

Page de statistiques : `http://10.0.10.2:8404/stats` (depuis vADMIN) — colonne *Status* et *LastChk*.

HAProxy — Répartition WordPress

Configuration

```
frontend fe_wordpress
    bind 10.0.10.2:80
    option forwardfor
    http-request set-header X-Forwarded-Proto http
    default_backend be_wordpress

backend be_wordpress
    balance roundrobin
    option httpchk
    http-check send meth GET uri /wp-login.php ver HTTP/1.1 hdr Host 10.0.10.2
    http-check expect status 200-399
    cookie SRV_WP insert indirect nocache httponly
    http-response set-header X-Served-By %[srv_name]
    default-server inter 3s fall 3 rise 2
    server web-01 10.0.10.1:80 check cookie web01
    server web-02 10.0.10.4:80 check cookie web02
```

Explication

Directive	Rôle
`mode http` (hérité de `defaults`)	HAProxy comprend HTTP (couche 7) : en-têtes, cookies, codes de retour
`option forwardfor`	Ajoute `X-Forwarded-For: <IP client>` ; Apache le lit via `mod_remoteip` pour journaliser la vraie IP
`balance roundrobin`	Alternance web-01 / web-02
`option httpchk` + `http-check send`	Health-check applicatif sur `/wp-login.php` : teste Apache **et** PHP **et** la base
`http-check expect status 200-399`	Un `302` (redirection) est sain ; un `500` ou `503` ne l'est pas
`cookie SRV_WP insert indirect nocache httponly`	HAProxy pose un cookie indiquant le serveur : un administrateur connecté reste sur le même serveur
`X-Served-By %[srv_name]`	Indique quel serveur a répondu : rend le failover **visible**

Pourquoi un health-check sur `/wp-login.php` ?

Check	Détecte Apache arrêté	Détecte PHP cassé	Détecte base inaccessible
TCP simple (`check` seul)	Oui	Non	Non
HTTP sur `/`	Oui	Oui	Partiellement (selon cache)
HTTP sur `/wp-login.php`	Oui	Oui	Oui (WordPress interroge la base)

Persistance de session

Sans persistance, un utilisateur connecté à `wp-admin` alterne entre web-01 et web-02. Comme les **clés et sels** de `wp-config.php` sont identiques sur les deux serveurs, la session reste valide ; la persistance n'est donc pas strictement nécessaire pour WordPress, mais elle :

- évite des incohérences pendant une synchronisation de fichiers ;
- facilite le diagnostic (un utilisateur = un serveur).

Si le serveur du cookie tombe, `option redispatch` envoie le client sur l'autre serveur.

Vérification

```
# SUR adm-01 — sans cookie : alternance
for i in 1 2 3 4; do curl -sI http://10.0.10.2 | grep -i x-served-by; done

# Avec cookie : persistance
curl -s -c /tmp/cj -o /dev/null http://10.0.10.2/
for i in 1 2 3; do curl -s -b /tmp/cj -o /dev/null -D - http://10.0.10.2/ | grep -i x-served-by;
done
```

Tests associés : T13 (Plan-de-tests.md), T14 à T16 (Plan-de-tests.md).

HAProxy — Publication de GLPI

Configuration

```
frontend fe_glpi
    bind 10.0.10.2:8080
    option forwardfor
    http-request set-header X-Forwarded-Proto http
    default_backend be_glpi

backend be_glpi
    option httpchk
    http-check send meth GET uri / ver HTTP/1.1 hdr Host 10.0.10.3
    http-check expect status 200-399
    http-response set-header X-Served-By %[srv_name]
    server glpi 10.0.10.3:80 check inter 5s fall 3 rise 2
```

Pourquoi publier GLPI via HAProxy alors qu'il n'y a qu'un serveur ?

Raison	Explication
Point d'entrée unique	Utilisateurs et règles pfSense ne connaissent que `10.0.10.2`

Raison	Explication
Évolutivité	Ajouter un GLPI-02 = ajouter une ligne `server`, sans toucher aux clients
Visibilité	État de GLPI visible dans la page de statistiques
Sécurité	vUSER n'a besoin d'aucun accès direct à glpi

Pourquoi le port 8080 ?

Le port 80 de HAProxy est déjà utilisé par WordPress. Deux solutions :

Solution	Configuration	Choix
Port distinct	`bind 10.0.10.2:8080`	**Retenu** : simple, sans DNS
Même port, routage par nom d'hôte	`acl is_glpi hdr(host) -i glpi.lab.local` + `use_backend be_glpi if is_glpi`	Nécessite un DNS interne (amélioration future)

Vérification

```
# SUR adm-01
./scripts/validation/check-glpi.sh
curl -sI http://10.0.10.2:8080/ | grep -Ei 'HTTP/|X-Served-By'
```

Configuration côté GLPI : MO-4-GLPI.md

HAProxy — Répartition SQL

Configuration

```
listen galera_cluster
    bind 10.0.10.2:3306
    mode tcp
    option tcplog
    balance roundrobin
    option mysql-check user haproxy_check post-41
    timeout client 30m
    timeout server 30m
    default-server inter 3s fall 3 rise 2 on-marked-down shutdown-sessions
    server bdd-01 10.0.20.1:3306 check
    server bdd-02 10.0.20.2:3306 check
    server bdd-03 10.0.20.3:3306 check
```

Explication

Directive	Rôle
`mode tcp`	HAProxy relaie les octets sans comprendre le protocole MySQL (couche 4)
`balance roundrobin`	Chaque **nouvelle connexion** va au nœud suivant. Une connexion ouverte reste sur son nœud
`option mysql-check user haproxy_check post-41`	Le health-check ouvre une vraie session MySQL (handshake + authentification)
`timeout client/server 30m`	Les connexions SQL peuvent rester inactives longtemps (pools, sessions GLPI)
`on-marked-down shutdown-sessions`	Quand un nœud est déclaré DOWN, ses sessions sont coupées : les clients se reconnectent ailleurs

Pourquoi passer par HAProxy plutôt qu'un nœud ?

- WordPress et GLPI n'acceptent **qu'une seule adresse** de serveur SQL.
- Si cette adresse était `10.0.20.1`, la panne de bdd-01 couperait les applications malgré les deux autres nœuds.
- HAProxy fournit une adresse **stable** (`10.0.10.2:3306`) et écarte automatiquement un nœud en panne.

Round-robin ou écrivain unique ?

Mode	Configuration	Avantage	Inconvénient
Round-robin (lab)	3 serveurs actifs	Démontre le multi-maître ; charge répartie	Deux écritures concurrentes sur la même ligne via deux nœuds → conflit de certification (`Deadlock`)
Écrivain unique (production)	1 actif + 2 `backup`	Aucun conflit de certification	Un seul nœud travaille ; les autres attendent

Le round-robin est conservé dans le lab pour sa valeur démonstrative (`test-sql-round-robin.sh`). Avec la charge très faible de WordPress et GLPI en lab, les conflits sont improbables. En production, la variante commentée dans `haproxy.cfg.example` (dépôt GitHub) serait préférable.

IP source et comptes SQL

HAProxy étant un proxy TCP, MariaDB voit toutes les connexions provenir de **10.0.10.2**. Les comptes applicatifs sont donc déclarés pour cet hôte :

```
-- SUR bdd-01
CREATE USER 'wordpress'@'10.0.10.2' IDENTIFIED BY '<PASSWORD>';
CREATE USER 'glpi'@'10.0.10.2'      IDENTIFIED BY '<PASSWORD>';
```

Conséquence de sécurité : MariaDB ne peut plus distinguer web-01 de glpi. Le filtrage par source est assuré **en amont** (pfSense : seul vWEB et vADMIN atteignent `10.0.10.2:3306`).

Vérification

```
# SUR haproxy
./scripts/utilities/test-sql-round-robin.sh -n 9
```

```
  connexion  1 -> mysql01
  connexion  2 -> mysql02
  connexion  3 -> mysql03
  ...
[OK]   3 nœuds distincts ont répondu (round-robin effectif)
```

(Format de sortie illustratif ; le résultat réel est consigné dans la matrice, test T10.)
